



Compiler Construction

Lab No. 7

Submitted By: Fatima Zahra

Sap Id: 55551

Submitted To: Ma'am Hasnat Bibi

Riphaah International University of Islamabad

Task 01

Converted to Non-Left Recursive Form:

$E \rightarrow T E'$

$E' \rightarrow - T E' \mid \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \varepsilon$

$F \rightarrow \text{id} \mid (E)$

Code:

```
#include <iostream>
#include <string>
#include <cstdlib>
using namespace std;
string input;
int indexPos = 0;
void S();
void match(char expected)
{
    if (indexPos < input.length() && input[indexPos] == expected)
    {
        indexPos++;
    }
    else
    {
        cout << "Syntax Error: Expected '" << expected << "' but found '" << input[indexPos] << "'";
        if (indexPos < input.length())
            cout << input[indexPos];
        else
            cout << "end of input";
    }
}
```

```

        cout << "\n";
        exit(1);
    }}
void S() // S ? a S b | e
{
    if (indexPos < input.length() && input[indexPos] == 'a')
    {
        match('a'); // Match 'a'
        S(); // Recursive call for the inner S
        match('b'); // Match 'b'
    }
}
int main()
{
    cout << "Enter expression (only a and b): ";
    cin >> input;
    // Validate input contains only 'a' and 'b' using traditional for loop
    for (int i = 0; i < input.length(); i++)
    {
        if (input[i] != 'a' && input[i] != 'b')
        {
            cout << "Invalid character in input. Only 'a' and 'b' allowed.\n";
            return 1;
        }
    }
    indexPos = 0;
    S();
    if (indexPos == input.length())
        cout << "String \"" << input << "\" Accepted\n";
}

```

```

else
    cout << "String \"\" << input << \"\" Rejected - Extra characters at end\n";
return 0;
}

```

Output:

```

C:\Users\TOSHIBA\Documents\Task 1.exe
Enter expression (only a and b): ab
String "ab" Accepted

-----
Process exited after 4.743 seconds with return value 0
Press any key to continue . . .

```

```

C:\Users\TOSHIBA\Documents\Task 1.exe
Enter expression (only a and b): aabb
String "aabb" Accepted

-----
Process exited after 3.694 seconds with return value 0
Press any key to continue . . .

```

```

C:\Users\TOSHIBA\Documents\Task 1.exe
Enter expression (only a and b): aaabbb
String "aaabbb" Accepted

-----
Process exited after 4.358 seconds with return value 0
Press any key to continue . . .

```

Explanation of How Recursion Matches the Symbols:

For input "ab":

1. Start $\rightarrow$ S()
2. At position 0: 'a' found $\rightarrow$ match('a'), call S()
3. At position 1: 'b' found (but S() sees no 'a') $\rightarrow$ ϵ production (do nothing)
4. Back to first S(), match('b') at position 1
5. All input consumed $\rightarrow$ Accepted

For input "aabb":

1. Start $\rightarrow$ S()
2. Position 0: 'a' $\rightarrow$ match('a'), call S()
3. Position 1: 'a' $\rightarrow$ match('a'), call S()

4. Position 2: 'b' found (no 'a') $\rightarrow \epsilon$ production
5. Back to second S(), match('b') at position 2
6. Back to first S(), match('b') at position 3
7. All input consumed $\rightarrow$ Accepted

For input "aaabbb":

1. S() calls itself recursively 3 times for each 'a'
2. Then matches 3 'b's in reverse order
3. The recursion creates nested structures: $a(a(a(\epsilon)b)b)b$

The recursion works like a stack - each 'a' pushes a call, each 'b' pops a call. The number of 'a's must equal the number of 'b's for the string to be accepted.

Task 02:

Convert Grammar to Non-Left Recursive Form and Implement

Original Grammar (Left Recursive):

$E \rightarrow E - T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id} \mid (E)$

Converted to Non-Left Recursive Form:

$E \rightarrow T E'$

$E' \rightarrow - T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow \text{id} \mid (E)$

Code:

```
#include <iostream>
```

```
#include <string>
```

```
#include <cstdlib>
```

```
using namespace std;
```

```
string input;
```

```
int indexPos = 0;
```

```

void E();
void Eprime();
void T();
void Tprime();
void F();
void match(char expected)
{
    if (indexPos < input.length() && input[indexPos] == expected)
    {
        indexPos++;
    }
    else
    {
        cout << "Syntax Error at position " << indexPos << ": Expected '" << expected << "'\n";
        exit(1);
    }
}

void E() // E ? T E'
{
    T();
    Eprime();
}

void Eprime() // E' ? - T E' | e
{
    if (indexPos < input.length() && input[indexPos] == '-')
    {
        match('-');
        T();
    }
}

```

```

    Eprime();
}
// else: e production
}
void T() // T ? F T'
{
    F();
    Tprime();
}
void Tprime() // T' ? * F T' | e
{
    if (indexPos < input.length() && input[indexPos] == '*')
    {
        match('*');
        F();
        Tprime();
    }
}
void F() // F ? id | (E)
{
    if (indexPos < input.length())
    {
        // Check for 'id' (two-character token)
        if (input[indexPos] == 'i' && indexPos + 1 < input.length() && input[indexPos + 1] == 'd')
        {
            // Match 'id' as two characters
            match('i');
            match('d');
        }
    }
}

```

```

    }
    else if (input[indexPos] == '('){
match('(');
        E();
        match(')');
    }else{
        cout << "Syntax Error: Expected 'id' or '(' at position " << indexPos << "\n";
        exit(1);
    } } else
{
    cout << "Syntax Error: Unexpected end of input\n";
    exit(1);
} }
int main()
{
    cout << "Enter expression (use 'id' for identifiers, and operators '-', '*'): ";
    cin >> input;
    for (int i = 0; i < input.length(); i++)
    {
        char c = input[i];
        if (c != 'i' && c != 'd' && c != '-' && c != '*' && c != '(' && c != ')')
        {
            cout << "Invalid character " << c << " in input. Allowed: i, d, -, *, (, )\n";
            return 1;
        }
    }
    indexPos = 0;
    E();

```

```

if (indexPos == input.length())
    cout << "String \"" << input << "\" Accepted\n";
else
    cout << "String \"" << input << "\" Rejected - Extra characters at end\n";

return 0;}

```

Output:

```

C:\Users\TOSHIBA\Documents\task 2.exe
Enter expression (use 'id' for identifiers, and operators '-','*'): id-id*id
String "id-id*id" Accepted

-----
Process exited after 11.52 seconds with return value 0
Press any key to continue . . .

```

Test for "id - id * id":

Step-by-step parsing:

1. $E() \rightarrow T() \rightarrow F() \rightarrow \text{match}('i','d') \rightarrow T'() \rightarrow \epsilon$
2. $E'() \rightarrow \text{match}('-') \rightarrow T() \rightarrow F() \rightarrow \text{match}('i','d') \rightarrow T'() \rightarrow \text{match}('*') \rightarrow F() \rightarrow \text{match}('i','d') \rightarrow T'() \rightarrow \epsilon$
3. $E'() \rightarrow \epsilon$
4. All input consumed $\rightarrow$ Accepted

Task 3:

Extend the parser to support operators:

$+ \ - \ * \ /$

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid - T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid / F T' \mid \epsilon$

$F \rightarrow \text{id} \mid (E)$

Test expressions:

id + id
(id + id) * id
id * id + id

Explain parsing steps.

Code:

```
#include <iostream>
#include <string>
#include <cstdlib>

using namespace std;

string input;
int indexPos = 0;

void E();
void Eprime();
void T();
void Tprime();
void F();
void match(char expected)
{
    if (indexPos < input.length() && input[indexPos] == expected)
    {
        indexPos++;
    }
    else
    {
        cout << "Syntax Error at position " << indexPos << ": Expected '" << expected << "'\n";
        exit(1);
    }
}

void E() // E ? T E'
```

```

{
    T();
    Eprime();
}
void Eprime() // E' ? + T E' | - T E' | e
{
    if (indexPos < input.length())
    {
        if (input[indexPos] == '+')
        {
            match('+');
            T();
            Eprime();
        }
        else if (input[indexPos] == '-')
        {
            match('-');
            T();
            Eprime();
        }
        // else: e production
    }
}
void T() // T ? F T'
{
    F();
    Tprime();
}

```

```
void Tprime() // T' ? * F T' | / F T' | e
```

```
{  
    if (indexPos < input.length())  
    {  
        if (input[indexPos] == '*')  
        {  
            match('*');  
            F();  
            Tprime();  
        }  
        else if (input[indexPos] == '/')  
        {  
            match('/');  
            F();  
            Tprime();  
        }  
    }  
}
```

```
void F() // F ? id | (E)
```

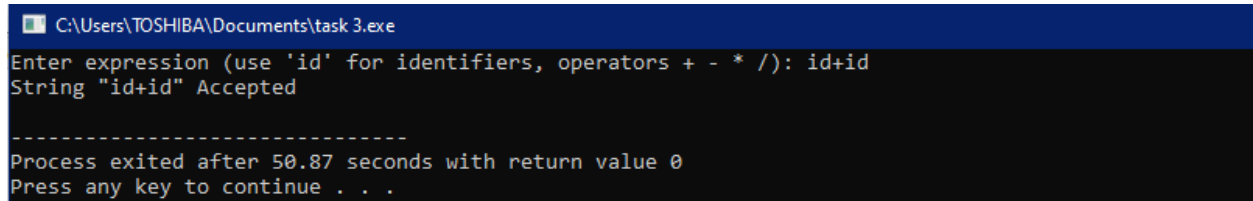
```
{  
    if (indexPos < input.length())  
    {  
        if (input[indexPos] == 'i' && indexPos + 1 < input.length() && input[indexPos + 1] == 'd')  
        {  
            match('i');  
            match('d');  
        }  
        else if (input[indexPos] == '(')  
        {
```

```

        match('(');
        E();
        match(')');
    }else
    {cout << "Syntax Error: Expected 'id' or '(' at position " << indexPos << "\n";
        exit(1);
    }}else{
        cout << "Syntax Error: Unexpected end of input\n";
        exit(1);
    }}
int main()
{
    cout << "Enter expression (use 'id' for identifiers, operators + - * /): ";
    cin >> input;
    indexPos = 0;
    E();
    if (indexPos == input.length())
        cout << "String \"" << input << "\" Accepted\n";
    else
        cout << "String \"" << input << "\" Rejected - Extra characters at end\n";
    return 0;
}

```

Output:



```

C:\Users\TOSHIBA\Documents\task 3.exe
Enter expression (use 'id' for identifiers, operators + - * /): id+id
String "id+id" Accepted
-----
Process exited after 50.87 seconds with return value 0
Press any key to continue . . .

```

```
C:\Users\TOSHIBA\Documents\task 3.exe
Enter expression (use 'id' for identifiers, operators + - * /): (id+id)*id
String "(id+id)*id" Accepted

-----
Process exited after 13.96 seconds with return value 0
Press any key to continue . . .
```

```
C:\Users\TOSHIBA\Documents\task 3.exe
Enter expression (use 'id' for identifiers, operators + - * /): id*id+id
String "id*id+id" Accepted

-----
Process exited after 10.14 seconds with return value 0
Press any key to continue . . .
```

Test Expressions and Parsing Steps:

1. Test: "id + id"

Parsing steps:

$E() \rightarrow T() \rightarrow F() \rightarrow \text{match('i','d')} \rightarrow T'() \rightarrow \epsilon$

$E'() \rightarrow \text{match('+')} \rightarrow T() \rightarrow F() \rightarrow \text{match('i','d')} \rightarrow T'() \rightarrow \epsilon \rightarrow E'() \rightarrow \epsilon$

Result: Accepted

2. Test: "(id + id) * id"

Parsing steps:

$E() \rightarrow T() \rightarrow F() \rightarrow \text{match('(')} \rightarrow E() \rightarrow T() \rightarrow F() \rightarrow \text{match('i','d')} \rightarrow T'() \rightarrow \epsilon$

$E'() \rightarrow \text{match('+')} \rightarrow T() \rightarrow F() \rightarrow \text{match('i','d')} \rightarrow T'() \rightarrow \epsilon \rightarrow E'() \rightarrow \epsilon$

$\text{match(')')} \rightarrow T'() \rightarrow \text{match('*')} \rightarrow F() \rightarrow \text{match('i','d')} \rightarrow T'() \rightarrow \epsilon$

$E'() \rightarrow \epsilon$

Result: Accepted

3. Test: "id * id + id"

Parsing steps:

$E() \rightarrow T() \rightarrow F() \rightarrow \text{match('i','d')} \rightarrow T'() \rightarrow \text{match('*')} \rightarrow F() \rightarrow \text{match('i','d')} \rightarrow T'() \rightarrow \epsilon$

$E'() \rightarrow \text{match('+')} \rightarrow T() \rightarrow F() \rightarrow \text{match('i','d')} \rightarrow T'() \rightarrow \epsilon \rightarrow E'() \rightarrow \epsilon$

Result: Accepted